Input, output and basic strings

How `print` lays out values, what a string is, escape characters, f-strings, and reading numbers typed by the user.

THE QUESTION

How does a program read what a person types, and how does it show results neatly?

THE INTUITION

`input()` hands back what was typed as text, even `120`. `print` writes values with spaces between and a new line after; an f-string builds the text first, numbers shaped as you ask.

KEY TAKEAWAY

Convert typed text with `int()` or `float()` before doing arithmetic. Shape numbers inside an f-string, as in `{x:.2f}`.

person types 120 → input() gives '120' → int() gives 120 → compute → f-string shapes the text → print shows it

1 `print` writes values, spaces and a new line

```
code                                  output
1 print("Total:", 3, 4.5)            Total: 3 4.5 \n
2 print("2 + 3 =", 2 + 3)           2 + 3 = 5 \n
3 print()                             \n
4 print("done")                      done \n
```

␣ sep, put between values (a space) `\n` end, put after the last value (a new line)

`print` writes the value of each **argument**. Between two values it puts a **separator** (`sep`, a space by default); after the last, an **end** (`end`, a **newline**: the invisible character that starts the next line). The spaces inside `"2 + 3 ="` belong to the text; only the one before `5` comes from `sep`. `print()` alone writes an empty line.

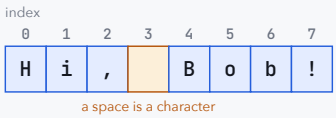
2 `sep` and `end` change the glue

code (s1 = "Hello", s2 = "World")	output
`print(s1, s2)`	`Hello World \n`
`print(s1, s2, sep="")`	`HelloWorld \n`
`print("Alpha", s1, s2, sep="<--->")`	`Alpha<--->Hello<--->World \n`
`print("Loading", end="")` `print("...")`	`Loading... \n`
`print("a", end=" | ")` `print("b")`	`a | b \n`

␣ sep ␣ end | an empty sep or end: nothing between

`sep` and `end` are **keyword arguments**: named, and placed after the values. `sep=""` glues values with nothing between; `end=""` keeps the next `print` on the same line.

3 A string is a row of characters



8 boxes, so len is 8; the last index is 7

```
>>> len("Hi, Bob!") 8
>>> len("") 0
>>> len(" ") 1
>>> len(str(9876543210)) 10
```

A **string** (type `str`) is a sequence of **characters**: letters, digits, spaces, punctuation. Each has a position, its **index**, counted from 0. `len(s)` counts the characters; `""` is the **empty string**. `len(str(x))` counts the digits of a number (the notebook's 29-digit one gives 29). Using indexes comes on the slicing and strings in depth sheets.

4 `+` joins strings, `*` repeats them



s1 + " " + s2: no space appears unless you add one

```
print(s1 + s2)          HelloWorld
print(s1 + " " + s2)    Hello World
print("-" * 20)         --------------------
print("ab" * 3)         ababab
```

number or text? the REPL shows text in quotes

```
"5" + "5" '55'    5 + 5 10        "5" * 3 '555'    5 * 3 15
```

On strings, `+` is **concatenation**: it joins them end to end, adding nothing between. `*` with a whole number repeats a string. On numbers the same symbols do arithmetic. `+` cannot mix a string and a number (a TypeError): convert one side first with `str()` or `int()`.

5 A backslash types what the keyboard cannot

you type in code	print shows	len
`"one\ntwo"`	one two `\n` starts a new line	7
`"a\tb"`	a → b tab: jump to the next stop	3
`"C:\\temp"`	C:\temp	7
`"She said \"hi\""`	She said "hi"	13
`'It\'s'`	It's	4
`'say "hi"'`	say "hi"	8

the backslash pair (rose) is one character; each quote style may hold the other

Inside quotes, a backslash starts an **escape sequence**, two keys for one character: `\n` newline, `\t` **tab** (a jump to the next column stop), `\\` a backslash, `\"` or `\'` a quote that does not end the string. Or pick the other quote style: `"It's"` needs no escape.

6 An f-string fills in values

```
name = "Alice"; marks = 87
print(f"{name} scored {marks} out of 100")
out   Alice scored 87 out of 100
print(f"{marks} + 5 = {marks + 5}")
out   87 + 5 = 92
```

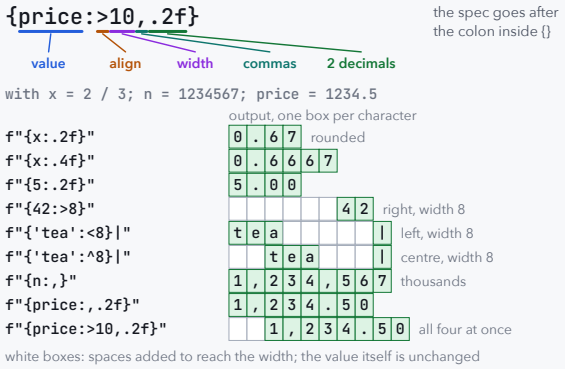
text outside {} is copied as typed; inside {} Python works out the value

An `f` right before the opening quote makes an **f-string** (formatted string). Each `{...}` holds an **expression**: Python works out its value, turns it into text and drops it in. No chains of `+` and `str()`, and you control how numbers look (next page).

Shape the output, convert the input

Decimals, widths and commas; `input()` returns text; `int()` and `float()` make numbers or raise a ValueError.

7 A format spec shapes a number



After a colon inside {} comes a **format spec**. `.2f` gives exactly 2 decimals, rounded (`f`: **fixed-point**). A number is the **width**, the fewest characters to fill, padded with spaces. < > ^ align left, right, centre (default: numbers right, text left). , groups thousands. The same width on every line lines up a column of numbers.

8 `input()` always returns a `str`

input.py

```
x = input("Enter a number: ")
print(type(x))
print(x * 2)
```

terminal

```
$ python3 input.py
Enter a number: 120
<class 'str'>
120120
```

you type 120 and press Enter; input() hands back the text



'120' is a str of three characters, so x * 2 repeats it: 120120

`input("prompt")` shows the **prompt**, **waits** for Enter, and returns what was typed as a string, without the Enter. Typed digits are still text.

9 Convert the text before doing arithmetic

line	code	x	y	terminal
1	`x = input("First: ")`	'10'		First: 10
2	`y = input("Second: ")`	'10'	'20'	Second: 20
3	`print(x + y)`	'10'	'20'	1020 joined text, not a sum
4	`x = int(x)`	10	'20'	
5	`y = int(y)`	10	20	
6	`print(x + y)`	10	20	30 a real sum

amber '10': a str (text) · blue 10: an int · amber box in the terminal: what you typed

The notebook's program, typed 10 and 20. Line 3 joins two texts; after `int()` the same + adds. Shorter: `int(input("First: "))`, and `float(input())` for decimals. `10,20` on one line: `x, y = input().split(",")` gives two strings (strings in depth sheet).

10 Text that is not a number gives a ValueError

these work

```
int("42") ⟶ 42                    int(" 42 ") ⟶ 42
float("4.5") ⟶ 4.5                float("7") ⟶ 7.0
int(float("4.99")) ⟶ 4             int(float("-4.99")) ⟶ -4
```

these stop the program: the last line of each error

```
int("4.5")       ValueError: invalid literal for int() with base 10: '4.5'
int("forty")     ValueError: invalid literal for int() with base 10: 'forty'
float("1,000")   ValueError: could not convert string to float: '1,000'
int("")          ValueError: invalid literal for int() with base 10: ''
```

A **ValueError**: the type is right (a string) but the text is not a number. `int()` wants whole-number text (spaces around are fine); `int(float(t))` drops decimals toward zero. Typing `twenty` at `int(input("Age: "))` stops the program with the same error, naming `'twenty'`.

11 Common mistakes: what you wrote, what you see, the fix

you wrote	typed	what you see: the error's last line, or the output	the fix
`print(input() + 1)`	19	`TypeError: can only concatenate str (not "int") to str`	`int(input())`
`print(input() + input())`	2, 3	`23` (no error, wrong output)	`int()` on each
`print(f"{input():.2f}")`	12.5	`ValueError: Unknown format code 'f' for object of type 'str'`	`float(input())`
`print("Sum: {2 + 3}")`		`Sum: {2 + 3}` (no error, wrong output)	add the `f`
`print('It's here')`		`SyntaxError: unterminated string literal (detected at line 1)`	`"It's here"`
`print("C:\new")`		`C: ⏎ ew` (no error, wrong output)	`"C:\\new"`
`print(len(2026))`		`TypeError: object of type 'int' has no len()`	`len(str(2026))`

Why: `input()` gives text; without the `f`, braces print as typed; in `"C:\new"` the `\n` is a newline; `len` needs a string.

12 Try it (answers in green)

1. What does `print("a", "b", "c", sep="-")` show?
   a-b-c
2. What is `len("Hi\tyou\n")`?
   7: each escape sequence is one character.
3. What is `"ab" * 3 + "c"`?
   ababab: * happens before +, as in arithmetic.
4. 5 typed at `x = input()`: `x * 3`? `int(x) * 3`?
   555 and 15.
5. Show 7 divided by 3 with 2 decimals.
   `f"{7 / 3:.2f}"` gives `2.33`.
6. Show 1234567.891 with commas and 2 decimals.
   `f"{1234567.891:,.2f}"` gives `1,234,567.89`.

sep and end set the glue · strings are rows of characters · input() gives a str · convert, then compute

Source: the strings and conditionals notebook (input and strings) and the variables and expressions notebook (output), nipunbatra.github.io/comp22. All outputs and errors come from running the code with Python 3.11.11, typed input fed through a terminal. Next: the conditionals sheet.